

PAYMENTS & DATA CUTOVER

Razorpay implementation and shared-database migration

Technical procedure, incident history, controls, verification and outstanding risks

Prepared from the internship repository, migration scripts, saved audit output and the operational evidence supplied in the project conversation. Status is as of 07 October 2026; this is not a fresh live-production audit.

Executive assessment

The internship app has a server-created Razorpay order, two signature-verified payment-confirmation channels (Checkout callback and webhook), and one shared payment recorder. It does not activate a new enrollment until a captured payment is recorded. The shared User/Enrollment migration committed for 310 retained legacy students and passed count and financial verification. Indexes were installed. Neither a live payment end-to-end test nor a coordinated production deployment of both apps is evidenced here.

Area	Verified state	Decision / caveat
Migration	335 users; 310 legacy students; 310 enrollments; billing ₹12,09,700; collected ₹9,52,800	Verification and independent audit passed; legacy Student retained.
Razorpay	Order creation, signature checks, provider fetch and one recorder are implemented	No live capture/retry/webhook proof; concurrency risk remains.
Release	Course app built and tested locally; internship code has local changes	Course Vercel deployment was previously blocked; coordinated rollout not confirmed.

Evidence standard

“Verified” means shown by a saved audit/verification output or directly by code. “Implemented” describes source behavior, not production behavior. “Unverified” means the report has no direct evidence. Counts may have changed after the saved audit. No credentials, student contact details or full migration-plan PII are reproduced.

1. Razorpay: current implementation

1.1 End-to-end procedure

1. The signed-in student submits a program and installment amount. POST /api/apply requires the student role, resolves the account from the session, loads the official Program price, checks first-installment bounds (₹500 through price), joining date and account phone, then creates or reuses one internship Enrollment for that user and offering. The new Enrollment starts payment_pending with no installments.
2. The route checks payment-attempt frequency (at most five tracked orders in ten minutes), acquires a unique RazorpayOrder lock for that enrollment, and creates the provider order server-side in INR paise. It stores the provider order ID and returns only checkout parameters to the browser. The order record is retained for audit; its expiry is 15 minutes and it is not TTL-deleted.
3. The browser opens Razorpay Checkout. A successful Checkout callback sends order ID, payment ID and signature to POST /api/verify. Independently, a payment.captured webhook can arrive at POST /api/razorpay/webhook. Both routes call recordRazorpayPayment; there is no second installment recorder.
4. The recorder retrieves the order and payment from Razorpay, requires captured status, matching order/amount/currency and stored order ownership notes. It appends an installment to the Enrollment, changes payment_pending to active, lets the Enrollment save hook derive collection/balance/fee status, marks RazorpayOrder processed, then attempts receipt email and admin notification.

1.2 Repeat enrollment and abandoned checkout

The unique (userId, type, offeringId) Enrollment key means clicking Enroll Now again for the same program reuses the unpaid Enrollment, while each attempt may create a new RazorpayOrder. The code expires a prior locally active, unpaid order before a new first-payment order; it does not delete the old order. A different program gets a different Enrollment. Paid enrollments reject a second first-payment form submission and direct the student to the dashboard balance flow. Dashboard/admin listing code currently hides payment_pending enrollments with no installment; those visibility edits are local worktree changes, so their deployment is unverified.

1.3 Existing protective controls

Control	Implementation	Limit
Price and identity	Program price, session user ID and ownership are resolved server-side; browser amount is bounded.	Balance path permits ₹1 minimum; review product policy.
Checkout signature	HMAC-SHA256 over stored orderId paymentId; timing-safe comparison.	Confirms callback authenticity, not settlement alone.
Webhook signature	HMAC over raw request body with separate webhook secret.	Webhook configuration and deliveries are not verified.
Capture check	Fresh provider order/payment fetch; captured, INR, IDs, amount, notes must match.	Reconciliation still needed for late/duplicate captures.
Database indexes	Unique sparse lockKey, orderId, paymentId; order records retained.	No unique transaction ID within embedded installments.

2. Razorpay: errors, residual risks and correct next implementation

2.1 What happened and how it was addressed

Observed problem	Action / current outcome
Before cutover, 15 historical RazorpayOrder records were present.	They were removed at the user's explicit request before migration. The original backup retains them; the frozen backup and latest audit show zero. No live payments were migrated.
Submitting an unpaid enrollment made it appear active in the student dashboard.	The Enrollment now begins payment_pending; local dashboard/admin queries exclude zero-installment pending records. Confirm deployed behavior with a test payment.
A cancelled Checkout can be retried.	A new tracked/provider order is created for the same unpaid Enrollment. Old local order is expired, not removed. This resolves repeated form use but needs late-capture reconciliation.

2.2 Open issues, ranked

Priority	Issue and consequence	Required correction
High	Callback and webhook can both read "not recorded" then save the same payment into the installment array. The check and append are not an atomic database operation; order update is separate.	Make financial recording transactional and concurrency-safe, with a unique payment claim/key. Test simultaneous callback/webhook and crash/retry paths.
High	Retry expires an earlier local order immediately even if its provider payment may still capture. Two captures could overpay; excess is only flagged required.	Define order-cancellation/reconciliation policy; reconcile every captured provider payment; build or document controlled refund workflow.
Medium	Receipt email is caught, but notification failure after accounting can return a 500 to the client.	Separate side effects from financial commit; make notification retryable and return recorded success once committed.
Medium	No evidenced end-to-end captured payment, webhook replay, duplicate delivery or refund test; latest audited order count is zero.	Exercise complete test-mode matrix and compare provider ledger with local installments before live release.
Medium	Earlier screenshot displayed a database connection credential. Rotation/revocation has not been confirmed.	Rotate exposed database credentials, update deployment secrets, and verify old credential no longer works.

2.3 Safe implementation and release sequence

1. Keep one recorder for both signed channels. Implement a single atomic payment claim and enrollment append in a MongoDB transaction, with a durable unique payment ID guarantee. Make retries return the existing receipt without adding another installment.
2. Decide policy for abandoned/late captures before releasing an old order lock. Never infer that a locally expired order cannot be captured. Reconcile every captured provider payment to exactly one installment or a documented refund exception.
3. Use test-mode keys and a configured test webhook. Test first payment, balance payment, failed/cancelled checkout, same-program retry, simultaneous callback/webhook, duplicated webhook, timeout after capture, overpayment and notification/email outage. Compare Razorpay payments to local order and installment records.
4. Deploy secrets through server-side configuration only (RAZORPAY_KEY_ID, RAZORPAY_KEY_SECRET, RAZORPAY_WEBHOOK_SECRET); keep the key secret and webhook secret out of client bundles and logs. Set live capture/webhook settings, monitor failures, and release only after reconciliation and rollback decisions are owned.

3. Shared User/Enrollment migration

3.1 Contract and intended data ownership

Both internship and course apps share one MongoDB database. User is a person identity with optional email/phone/password and permanent studentId when assigned. Enrollment is a program or certificate relationship owned by userId. Internship/course require an offering; certificate has a certificate number and no offering. A new online enrollment starts payment_pending. The legacy Student model and its documents remain available; they were not dropped. RazorpayOrder binds an exact user and enrollment to one payment attempt.

Collection	Key indexes / behavior
users	Partial unique indexes on string email, phone and studentId; studentId immutable; password hook hashes changed values.
enrollments	Partial unique userId/type/offeringId and certificateNumber; query indexes for offering/status/joinedAt and user/status/joinedAt; offering-versus-certificate validation; installment-derived totals.
razorpayorders	Unique sparse lockKey, orderId, paymentId; user/enrollment/time query indexes; audit-retained, no TTL.

3.2 Frozen state and preparation

1. Pause writes in both applications. The original pre-migration dump preserved 312 legacy students, 29 users and 15 Razorpay orders. At the user's direction, two assessment-cleared legacy records, two duplicate users and all 15 order records were removed; original backup is the recovery source for those records. A post-cleanup and frozen dump were then taken. The frozen dump has a SHA-256 manifest with 29 entries.
2. Run the read-only audit and dry-run planner against the internship database. The frozen working set was 27 users, 310 legacy students, zero enrollments and zero orders. The approved classification was 275 internship, 19 course and 16 certificate; 308 Users to create, two existing Users to reuse; zero manual-review cases.
3. Inspect the plan, identity matching and financial totals. The corrected plan omitted only two known placeholder addresses shared by 15 different students (identified by distinct phones); those students stayed separate. No actual email or phone is printed in this report. The final plan was student-enrollment-dry-run-2026-10-06T12-33-15.219Z.json.
4. Install the reviewed partial/unique indexes, then apply the approved plan with --apply. This order was necessary because an older non-partial email index would reject optional-email Users. The apply script uses one MongoDB transaction with snapshot read concern, majority write concern, count guard and deterministic IDs/upserts.
5. Run migration verification and independent audit before restoring writes or releasing both compatible applications. Later, the joinedAt repair synchronized 310 enrollment dates from the legacy joining-date field, with zero invalid dates. A second preview reported zero remaining updates.

4. Migration incident log and resolution

Failure / finding	Cause	Resolution and proof
mongodump not recognized; mongosh unavailable	Database tools were not installed/on PATH; a shell command was first entered in mongosh.	Ran mongodump from PowerShell after obtaining tools. Backup files and SHA-256 manifest were confirmed. App-side Node scripts replaced the need for mongosh checks.
MONGODB_URI is not set	A new PowerShell session lacked the process environment variable.	Set it for the command session. Do not paste or log the URI; rotate previously exposed credential.
Phone index refused	One User had phone as an empty string, which the partial string index includes.	Guarded cleanup unset that optional field; follow-up audit found zero empty string phones; index step retried.
NamespaceNotFound: enrollments	Index installer attempted listIndexes before the new collection existed.	Installer was adjusted to handle an absent collection; shared indexes then installed and MongoDB verified them.
E11000 email duplicate on apply	A placeholder email was shared by distinct students; unique email index correctly rejected it.	Transaction rolled back: audit still showed 27 users/zero enrollments. Planner omitted the two approved placeholders for 15 students and preflight checks unique planned values. Replanned and committed.
Admin list sorted by creation, not joining	Migrated joinedAt initially followed legacy createdAt rather than date of joining.	Transactional sync updated 310 joinedAt fields; repeat preview found zero; audit and migration verification passed.

4.1 Why the first failed apply did not partly migrate

The duplicate-key exception occurred inside the MongoDB transaction. A post-failure read-only audit found 27 Users and zero Enrollments, matching pre-apply state. The corrected plan was a new reviewed artifact; the original failed plan was not simply rerun.

4.2 Idempotency scope

The migration gives new Users and Enrollments deterministic IDs derived from the legacy Student ID, uses \$setOnInsert, and checks source counts. That makes repeating the same plan less likely to duplicate records. It is not a substitute for a post-run audit, especially if source data or the plan changes. The index installer is a separate, non-transactional operation and was independently verified.

5. Verified migration outcome and remaining release gates

5.1 Reconciliation at the saved audit

Measure	Before apply	After apply / 07 Oct audit
Users	27	335 (=27 + 308)
Legacy Students	310	310 (retained)
Enrollments	0	310
Razorpay orders	0	0
Total billing	₹12,09,700 legacy	₹12,09,700 enrollment
Total collected	₹9,52,800 legacy	₹9,52,800 installment-derived
Outstanding	₹2,56,900 implied	₹2,56,900 enrollment
Integrity	0 legacy financial mismatches	0 invalid enrollments; 0 duplicate User email/phone/studentId; 0 empty optional strings

Saved verification returned verified: true, checking counts, per-enrollment installment/derived totals, user links and assigned student IDs. The later read-only audit was generated at 2026-10-07 04:16:11 UTC. This is evidence of that point-in-time database state, not a present-time query.

5.2 Open operational issues

- Coordinated production cutover is not evidenced. The course app's local production build and 13 tests passed, but a Vercel deployment was previously blocked; the internship app has local uncommitted changes. Check the actual deployed revisions and configuration before reopening writes.
- The old Student model and documents remain intentionally. Ensure neither deployed app writes legacy Student as a parallel source of truth; later removal needs a separate approved cleanup and backup window.
- Ten legacy application records remain in the database. The migration script handled Users and Enrollments, not dependent application references; inspect whether any active course/employer/placement route still depends on old Student IDs before release.
- Two assessment-cleared legacy records were removed by explicit request before the 310-record migration. They are not included in these counts and require an explicit restore/product decision if needed; original backup retains them.
- No restore drill, live payment reconciliation or post-deployment smoke-test result is evidenced. Do not infer payment readiness from database verification alone.

5.3 Controlled next steps

1. Freeze and compare release commits in both repositories; confirm identical shared schemas and indexes, environment variables and deployed routes. Resolve the blocked course deployment.
2. Fix the payment-recorder concurrency risk and late-capture/refund procedure; add test-mode integration and reconciliation tests before live payment traffic.
3. Run a fresh read-only audit immediately before and after release, confirm baseline counts/totals, and execute both-app smoke tests for sign-in, first paid enrollment, balance payment, dashboard/admin visibility and course enrollment. Keep backups and a documented rollback owner.

6. Evidence and references

Repository evidence

- Payment creation: `src/app/api/apply/route.ts`; browser checkout: `src/components/RazorpayCheckout.tsx` and `src/app/apply/page.tsx`.
- Payment confirmation: `src/app/api/verify/route.ts`; `src/app/api/razorpay/webhook/route.ts`; single recorder: `src/lib/record-razorpay-payment.ts`.
- Schemas: `src/models/user.ts`; `src/models/Enrollment.ts`; `src/models/RazorpayOrder.ts`.
- Migration: `scripts/plan-student-enrollment-migration.mjs`; `apply-student-enrollment-migration.mjs`; `verify-student-enrollment-migration.mjs`; `install-shared-indexes.mjs`; `audit-shared-database.mjs`; `sync-enrollment-joined-at.mjs`.
- Saved output: `audit-output/shared-database-audit-2026-10-07T04-16-10.118Z.json`; corrected plan: `audit-output/student-enrollment-dry-run-2026-10-06T12-33-15.219Z.json`.
- Architecture map: `docs/PROJECT_KNOWLEDGE.md` and `docs/STUDENT_ENROLLMENT_ARCHITECTURE.md`. Some map text still describes the pre-migration state; source code and saved audit take precedence for current status.

External provider guidance

Razorpay Integration Steps (server order, signature verification, captured status, test mode):

<https://razorpay.com/docs/server-integration/python/test-app/>

Razorpay Trust Portal checklist (trusted stored order ID, signature and secret handling):

<https://razorpay.com/security/checklist/>

Razorpay documentation was consulted for provider requirements. The implementation/risk findings in this report are deductions from the repository and the saved migration evidence, not a certification by Razorpay.

Report boundary

No data was modified to produce this report. It does not certify a production deployment, an actual captured payment, a credential rotation, a successful restore drill or the state of the shared database after the saved audit timestamp.